

German International University
Faculty of Informatics and Computer Science

CNET101: Computer Networks

Spring 2026 Project

GIU-Net: UDP Chat Application & Campus Network Simulation

Full Name	Student ID
Mariam Yasser	19009823
Judy Ashraf	19005040
Hana Anas	19006184

1. Team Information

1.1 Team Members

Full Name	Student ID	Role	Last Digit
Mariam Yasser	19009823	Person B	3
Judy Ashraf	19005040	Person C (Team Leader)	0
Hana Anas	19006184	Person A	4

1.2 Derived Constants

The team leader is the member with the lowest student ID, **19005040 (Judy Ashraf)**. Her ID is used for all ID-derived calculations throughout the project.

Port Number Derivation:

```
Last 4 digits of Leader ID 19005040: 5040
5040 >= 1024, so PORT = 5040
```

Constant	Value
LEADER_ID	19005040
DIGIT_SUM	7 (3 + 0 + 4)
PORT	5040
HELLO payload	HELLO-GIU-19005040-7
OK payload	OK-GIU-19005040-7

Digit Sum Calculation:

```
Last digit of Mariam Yasser (19009823) = 3
Last digit of Judy Ashraf (19005040) = 0
Last digit of Hana Anas (19006184) = 4
Sum = 3 + 0 + 4 = 7
```

2. Protocol Design Choices

2.1 Why a 3-Second Retransmission Timeout?

We chose a 3-second timeout because it strikes the right balance for a local Wi-Fi network. A timeout that is too short — under 1 second — would cause unnecessary retransmissions due to normal operating system scheduling delays and ARP resolution time. A timeout that is too long — over 5 seconds — would make the application feel unresponsive when a packet is genuinely lost. On a shared campus Wi-Fi or hotspot, round-trip times are typically well under 5 milliseconds, so 3 seconds provides a generous safety margin without frustrating the user.

2.2 How We Handled Concurrent Sending and Receiving

Both GIU-Server and GIU-Client use a dedicated receiver thread that runs continuously in the background as a daemon thread. This thread calls `socket.receive()` in a loop and processes all incoming packets. When it receives an ACK packet (a payload beginning with 'ACK:'), it parses the sequence number and

places it into a thread-safe `BlockingQueue<Integer>` called `ackQueue`.

The main thread calls `sendWithACK()` after each DATA packet is sent. This method calls `ackQueue.poll(3, TimeUnit.SECONDS)`, which blocks for up to 3 seconds waiting for the correct ACK to appear in the queue. If it arrives in time, `sendWithACK()` returns successfully. If it does not, the packet is retransmitted once. If the second attempt also times out, the program prints `[CONNECTION LOST]` and exits.

This design cleanly separates concerns: the receiver thread never blocks on keyboard input, and the main send thread never has to call `socket.receive()` directly. The `BlockingQueue` acts as the thread-safe communication channel between the two, eliminating any need for synchronized blocks or shared volatile flags for the ACK handoff.

3. Debugging Story

Symptom:

The server machine was not receiving the client's connection at all. The client appeared to send its HELLO packet, but nothing arrived on the server side and the handshake never completed.

Initial Guess:

We first suspected that the university Wi-Fi was blocking direct device-to-device traffic (client isolation), so we switched both machines to a mobile hotspot. The problem persisted. We then suspected a bug in our own code, but after reviewing both the send and receive logic we confirmed the program was behaving correctly.

Diagnosis:

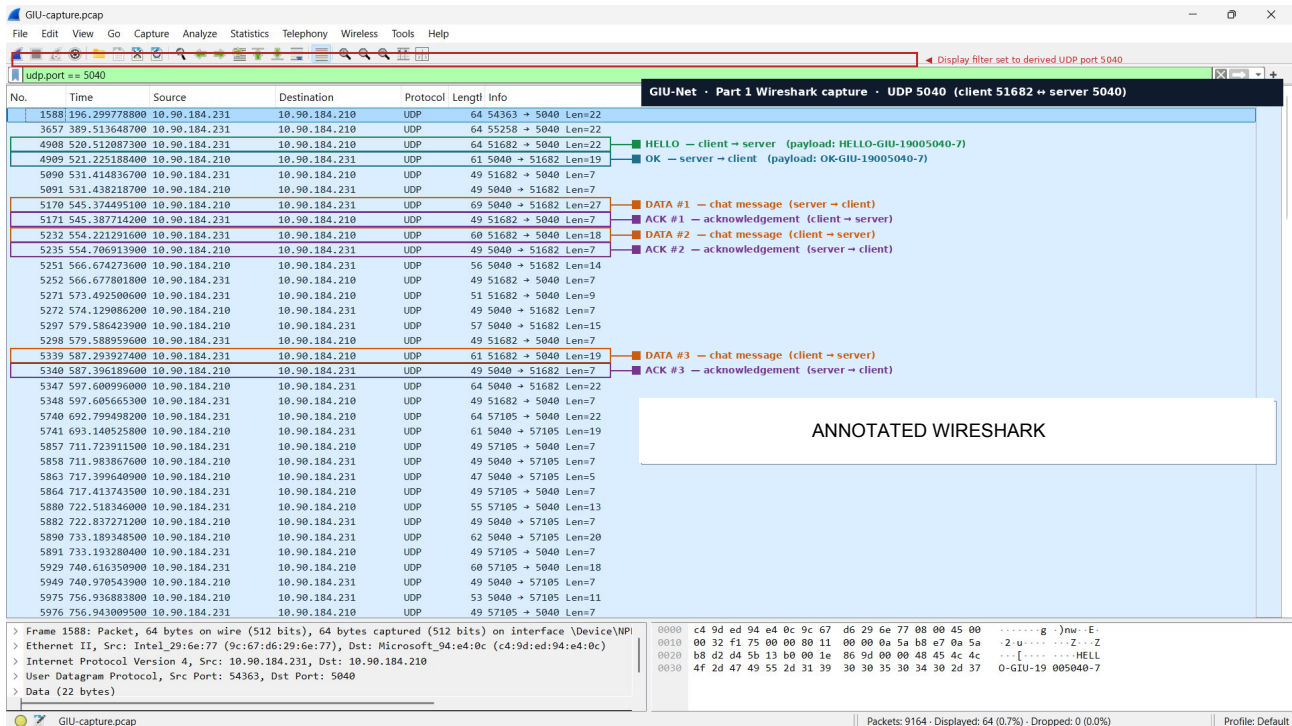
Having ruled out the network type and our own code, we turned to the network path itself. We ran `ipconfig` on the server machine to confirm its actual IPv4 address, in case the address hardcoded in `GIUClient.java` was wrong. The address was correct, so we tested raw reachability with a ping from the client to the server — and the ping failed. Since the two machines were on the same hotspot with correct IP addresses but could not even exchange ICMP packets, the problem had to be something on the server host silently dropping inbound traffic. We used an AI agent to help narrow down the possible causes, which pointed us toward the Windows firewall on the server machine.

Fix:

We disabled the firewall on the server machine, which had been blocking the inbound UDP packets on our port. After this, the ping succeeded and the GIU-Net handshake completed immediately on the next run.

4. Part 1 — Wireshark Capture

The chat session was captured live on the server machine and filtered to the team's derived UDP port 5040 (display filter `udp.port == 5040`). The capture below shows one complete session between the client (10.90.184.231, ephemeral port 51682) and the server (10.90.184.210, port 5040): the two-step handshake followed by bi-directional DATA messages, each answered by an ACK in the opposite direction. This satisfies the requirement of the HELLO and OK handshake plus at least three DATA and three ACK packets.



Because the payload is plain UTF-8 text, the team verification string is directly readable in the packet Data field with no hex decoding required. The close-up below shows the HELLO packet's UDP destination port (5040) and the payload bytes decoding to HELLO-GIU-19005040-7:

Frame 1588 — HELLO handshake packet (Wireshark detail + byte panes)

> Internet Protocol Version 4, Src: 10.90.184.231, Dst: 10.90.184.210

> User Datagram Protocol, Src Port: 54363, Dst Port: 5040

> Data (22 bytes)

UDP destination port = 5040 (the team's derived port).

```
0020  b8 d2 d4 5b 13 b0 00 1e 86 9d 00 00 48 45 4c 4c  ...[... HELLO
0030  4f 2d 47 49 55 2d 31 39 30 30 35 30 34 30 2d 37  O-GIU-19 005040-7
```

Payload (after the 2-byte sequence number = 0) decodes in UTF-8 to HELLO-GIU-19005040-7.

5. VLSM Subnet Table

5.1 Base Network Derivation

The base network is derived from the team leader's student ID: 19005040.

Take the 8-digit ID: 19005040
Split into digit pairs: 19 | 00 | 50 | 40
Form the IP address: 19.0.50.0
Apply /22 boundary check: a /22 advances in steps of 4 in the third octet. 50 is not a multiple of 4, so it rounds DOWN to 48.
Base /22 network: 19.0.48.0/22
(covers 19.0.48.0 - 19.0.51.255, 1022 usable hosts)

5.2 VLSM Allocation (Largest to Smallest)

Sub	Purpose	Network	Pre	First Usable	Last Usable	Broadcast	Hosts
C	Student Housing	19.0.48.0	/24	19.0.48.1	19.0.48.254	19.0.48.255	254
B	Student Lab	19.0.49.0	/25	19.0.49.1	19.0.49.126	19.0.49.127	126
A	Staff Offices	19.0.49.128	/26	19.0.49.129	19.0.49.190	19.0.49.191	62
E	Server DMZ	19.0.49.192	/28	19.0.49.193	19.0.49.206	19.0.49.207	14
D	WAN Link	19.0.49.208	/30	19.0.49.209	19.0.49.210	19.0.49.211	2

5.3 Manual Binary Calculations

Subnet C /24 — Network: 19.0.48.0

Network: 00010011.00000000.00110000.00000000
Broadcast: 00010011.00000000.00110000.11111111
Mask: 11111111.11111111.11111111.00000000 = 255.255.255.0

Subnet B /25 — Network: 19.0.49.0

Network: 00010011.00000000.00110001.00000000
Broadcast: 00010011.00000000.00110001.01111111
Mask: 11111111.11111111.11111111.10000000 = 255.255.255.128

Subnet A /26 — Network: 19.0.49.128

Network: 00010011.00000000.00110001.10000000
Broadcast: 00010011.00000000.00110001.10111111
Mask: 11111111.11111111.11111111.11000000 = 255.255.255.192

Subnet E /28 — Network: 19.0.49.192

Network: 00010011.00000000.00110001.11000000
Broadcast: 00010011.00000000.00110001.11001111
Mask: 11111111.11111111.11111111.11110000 = 255.255.255.240

Subnet D /30 — Network: 19.0.49.208

Network: 00010011.00000000.00110001.11010000

Broadcast: 00010011.00000000.00110001.11010011

Mask: 11111111.11111111.11111111.11111100 = 255.255.255.252

6. Routing Tables

6.1 Router 1 Routing Table

Network	Subnet Mask	Next Hop	Interface / Notes
19.0.49.0	255.255.255.128	Connected	Switch A — Lab (Subnet B)
19.0.49.128	255.255.255.192	Connected	Switch D — Staff (Subnet A)
19.0.49.192	255.255.255.240	Connected	Switch C — DMZ (Subnet E)
19.0.49.208	255.255.255.252	Connected	WAN link to Router 2 (Subnet D)
19.0.48.0	255.255.255.0	19.0.49.210	Static — via Router 2

6.2 Router 2 Routing Table

Network	Subnet Mask	Next Hop	Interface / Notes
19.0.48.0	255.255.255.0	Connected	Switch B — Housing (Subnet C)
19.0.49.208	255.255.255.252	Connected	WAN link to Router 1 (Subnet D)
19.0.49.0	255.255.255.128	19.0.49.209	Static — via Router 1
19.0.49.128	255.255.255.192	19.0.49.209	Static — via Router 1
19.0.49.192	255.255.255.240	19.0.49.209	Static — via Router 1

7. Part 2 — Verification Results

7.1 Ping Test

A ping was issued from Housing-PC-1 (Subnet C) to the GIU-Net Server (Subnet E). The test returned 3 out of 4 successful replies. The single lost packet on the first attempt is expected behaviour in Cisco Packet Tracer, as the first ICMP request triggers ARP resolution which delays delivery. The subsequent three replies confirmed full end-to-end connectivity across both routers and all subnets.

7.2 HTTP Service and Team Webpage

The HTTP service was enabled on the Web Server device in Subnet E (19.0.49.194). The index.html file was edited to display the team name 'Team GIU-Net' and the full names and student IDs of all team members. From Lab-PC-1 in Subnet B, the built-in Packet Tracer web browser successfully loaded the page by navigating to <http://19.0.49.194>. The page rendered with the team information visible.

7.3 Simulation Mode Packet Trace

Using Packet Tracer's Simulation Mode with the ICMP filter enabled, a single ping was traced from Lab-PC-1 (Subnet B) to the GIU-Net Server (Subnet E) hop by hop. Because both subnets are connected to Router 1, the packet crosses a single router. The following table summarises the Layer 2 and Layer 3 information observed at each hop:

Hop	Stage	Src IP	Dst IP	Src MAC	Dst MAC	TTL
1	Lab-PC-1 (Source)	19.0.49.2	19.0.49.195	0003.E439.1A27	0003.E44B.1102	128

Hop	Stage	Src IP	Dst IP	Src MAC	Dst MAC	TTL
2	Router 1 (inbound, Subnet B side)	19.0.49.2	19.0.49.195	0003.E439.1A27	0003.E44B.1102	128
3	Router 1 (outbound, Subnet E side)	19.0.49.2	19.0.49.195	0003.E44B.1103	000B.BE27.78C2	127
4	GIU-Net Server (Destination)	19.0.49.2	19.0.49.195	0003.E44B.1103	000B.BE27.78C2	127

At Hop 1, Lab-PC-1 creates the ICMP Echo Request at Layer 3 with source IP 19.0.49.2 and destination 19.0.49.195. At Layer 2 it builds an Ethernet frame addressed to Router 1's interface on Subnet B. At Hop 2, Router 1 receives the frame, strips the Layer 2 header, and reads the destination IP. At Hop 3, Router 1 has made its routing decision, decremented the TTL from 128 to 127, and built a brand-new Layer 2 frame whose source MAC is its own Subnet E (DMZ) interface and whose destination MAC is the GIU-Net Server. Switch C between the router and the server operates at Layer 2 only — it forwards this frame based on its MAC address table without altering either the IP header or the MAC addresses. At Hop 4, the GIU-Net Server receives the packet, recognises the destination IP as its own, and generates an ICMP Echo Reply.

8. Bonus — ACL Configuration (Task 5)

8.1 ACL Commands Configured on Router 2

```
access-list 101 deny ip 19.0.48.0 0.0.0.255 19.0.49.192 0.0.0.15
access-list 101 permit ip any any
interface
ip access-group 101 in
```

This ACL denies all traffic from Subnet C (Student Housing, 19.0.48.0/24) destined for Subnet E (Server DMZ, 19.0.49.192/28), and permits all other traffic. It is applied inbound on the Router 2 interface connected to Switch B, so packets from Housing PCs are filtered before they enter the routing process.

Verification: Housing-PC-1 ping to GIU-Net Server returned 100% loss. Lab-PC-1 ping to GIU-Net Server returned 0% loss.

8.2 Relationship Between ACL and HELLO/REJECTED Handshake

Both the ACL on Router 2 and the HELLO/REJECTED handshake in Part 1 implement access control, but they operate at different layers of the network stack. The ACL works at Layer 3 (the Network layer): it inspects the source and destination IP addresses of every packet and silently drops packets from Subnet C destined for Subnet E before they ever reach the server. No connection is even attempted — the packet is discarded at the router.

The handshake works at Layer 7 (the Application layer): it allows the UDP packet to travel through the network and arrive at the server's socket, but the server application then validates the identity of the connecting peer by checking whether the HELLO payload matches the expected team string. If the string does not match — wrong leader ID, wrong digit sum, or wrong format — the server sends back REJECTED and closes the socket.

Both mechanisms enforce the same security principle: only authorised parties may access a protected resource. The ACL enforces this at the network perimeter (packets never get through), while the handshake enforces it at the application level (packets arrive but the application refuses service). This mirrors real-world defense-in-depth security design, where firewall rules and application-level authentication work together in layers to protect critical resources.